

Self Balancing Robot **Report**

Group 1 - ELEC 391
June 15, 2026

Table of Contents

Table of Contents	1
Peer Evaluations.....	2
System Overview	2
Final Design and Implementations	3
Mechanical Design	3
Electric Design	5
Software and Computer Design:	8
Design Evolution and Engineering Decisions	15
Velocity Control to Encoder Position Control	15
Straight-Line Movement	18
Turning Control	22
Ramp Development.....	25
Testing, Tuning, and Performance Analysis	29
Test 1: Motor Noise Test and Complementary Filter Tuning	29
Test 2: Single Loop PID Control Tuning	34
Test 3: Control Loop Frequency Test.....	36
Challenges and troubleshooting.....	39
Challenge 1: Hardware Damage.....	39
Challenge 2: Unequal Wheel Speed	41
Challenge 3: Z-Axis Gyroscope Bias	43
Conclusions	45
Future Improvements.....	45
References:.....	46
Appendix:	47

Group 1

Group Member Name
Russel Chan
How Yee Chaw
Matthew Wu
Mohammad Ghazi

System Overview

The final self-balancing robot uses a closed-loop control system to maintain balance while responding to remote movement commands. The Arduino Nano 33 BLE Sense Rev2 acts as the main controller. It reads the onboard IMU to estimate the robot's tilt angle, reads the wheel encoders to estimate wheel motion, and sends PWM commands to the motor drivers.

The robot balances using the inverted pendulum principle. When the body tilts forward or backward, the controller drives the wheels in the same direction as the fall to move the base back under the center of mass. The tilt angle is estimated using a complementary filter, which combines accelerometer and gyroscope measurements. This filtered angle is then used by the angle controller to determine the motor correction needed to keep the robot upright.

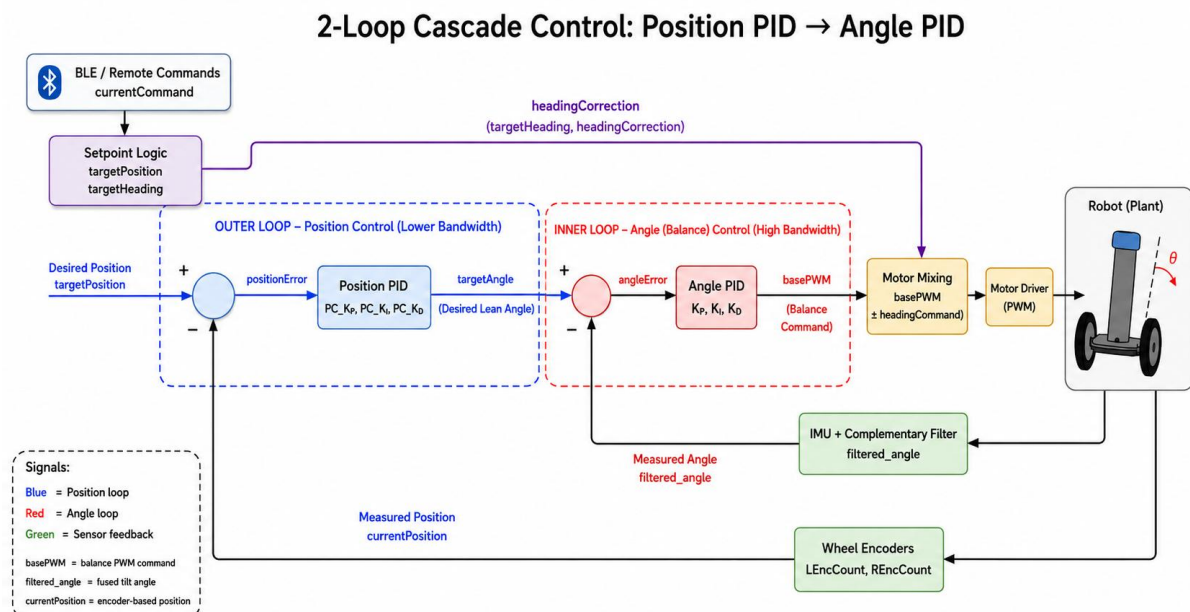


Figure 1: Block Diagram of our Self Balancing Robot

The overall control structure of our robot shown in Fig. 1. The system uses a double cascade control loop, where the outer position loop uses encoder feedback to generate a target lean angle, and the

inner angle loop uses the filtered IMU angle to generate the final motor PWM command. This allows the robot to control slower position changes while still maintaining a fast balancing response.

The BLE system provides the high-level movement commands for the robot. In this setup, the Android phone acts as the BLE central device, while the Arduino Nano acts as the BLE peripheral device. The phone connects to the Arduino through the remote control app and writes command values to a BLE characteristic. These commands are interpreted by the Arduino as movement states such as forward, backward, left, right, stop, or ramp mode. The BLE commands do not directly drive the motors, instead, they update the target position, heading, or operating mode used by the control system.

Final Design and Implementation

Mechanical Design:

The mechanical structure of the self-balancing robot was built around a provided two-wheel chassis, which served as the starting platform for the project. Since the chassis was supplied, our design work focused primarily on how to mount the electrical and control components onto the frame in a way that supported stable balancing, easy wiring, and a reasonable center of mass. The final mechanical design is shown in the attached front and side views.



Figure 2: Front View of Self Balancing Robot



Figure 3: Side View of Self Balancing Robot

The robot uses a two-wheeled differential-drive layout, with one DC gearmotor attached to each side of the chassis and a wheel mounted directly to each motor shaft. This configuration is appropriate for a self-balancing robot because the wheels act as the pivot point, allowing the body to behave like an inverted pendulum. The chassis itself consists of a compact layered structure with a lower motor-support region and an upper mounting platform for the electronics.

The frame includes rigid side supports and vertical threaded rods, which provide mechanical strength and also create mounting space above the main body. These vertical members help keep the structure stiff while allowing enough room to place electronics, route wiring, and make adjustments during testing. Since the robot needed to survive repeated balancing tests and occasional falls, maintaining a rigid frame was important so that the sensor readings and controller behaviour were not affected by unwanted flexing of the structure.

Component placement was chosen to support three goals:

1. keep the robot mechanically stable,
2. maintain a usable center of mass for balancing
3. make the electrical system accessible for testing and debugging.

The main components were arranged as follows:

- Motors and wheels: mounted symmetrically on the left and right sides of the chassis to provide balanced drive torque and equal support.
- Battery pack: mounted low and near the middle of the robot body, between the two motors. This location protected the battery and helped keep the mass centered from left to right.
- Controller and prototyping electronics: mounted on the upper platform, for easy debugging.
- Vertical rods/standoffs: extended upward from the main body, contributing additional height and helping position the upper structure.

Center-of-mass placement was one of the most important mechanical considerations in the design of the robot. A self-balancing robot behaves as an inverted pendulum, so the location of the center of mass strongly affects how easy the robot is to balance.

In general, the center of mass must lie above the wheel axle for the robot to behave as a balancing system rather than a purely rolling platform. If the mass is too low, the balancing effect becomes weak. If it is too high or too far from the centerline, the robot can become mechanically unstable or oscillate excessively. Our provided design therefore aimed to keep the center of mass:

- above the wheel axle, so the robot would act as an inverted pendulum,
- centered laterally between the two wheels, so the left and right sides behaved as symmetrically as possible,
- close to the longitudinal center of the chassis, so forward and backward responses remained consistent.

The heavier items, particularly the motors and battery, were mounted near the middle of the chassis, while lighter electronics were placed on the upper platform. This gave the robot enough height to balance effectively, while still avoiding an excessively high center of mass. The relatively tall upper structure, including the threaded rods and mounted electronics, also increased the pendulum height slightly, which can make the balancing motion slower and easier for the controller to react to during testing.

Another important consideration was left-right symmetry. If one side of the robot were significantly heavier than the other, the motors would have to work unevenly and the robot would tend to drift or lean during operation. For this reason, the main components were arranged as symmetrically as possible, and wiring was routed so that it did not create unnecessary imbalance.

Due to the fact that the chassis was provided, the project focused less on fabricating a new frame and more on integrating the available structure into a functional self-balancing system. The final arrangement represents a compromise between stability, accessibility, symmetry, and ease of implementation.

Electrical Design:

The electrical system of the self-balancing robot was designed to connect the sensing, control, and actuation components into one integrated system. The main electrical components were the Arduino Nano 33 BLE Sense Rev2 microcontroller, two DRV8871 motor drivers, two 12 V Pololu DC gearmotors with integrated quadrature encoders, a battery supply, and a breadboard used for distributing signals and organizing connections. A top-down view of the final wiring layout is shown in Figure 4.

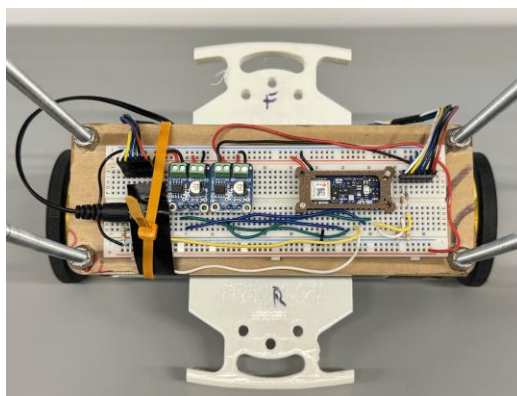


Figure 4: Final Electrical and Wiring Layout of Self Balancing Robot

The Arduino Nano was used as the main controller. It was responsible for reading the inertial measurement unit, processing encoder signals, running the balance controller, and sending PWM signals to the motor drivers. This board was selected because it includes an onboard BMI270 inertial measurement unit, which contains a 3-axis accelerometer and 3-axis gyroscope. These sensors were required to estimate the robot's tilt angle in real time. The board also supports Bluetooth Low Energy, which allowed the robot to be remotely controlled during the final demo.

The two DC motors were controlled using separate DRV8871 motor driver boards. Each DRV8871 acts as a MOSFET-based H-bridge, allowing the Arduino to control both the speed and direction of one motor. The Arduino cannot power the motors directly because the motors require much higher current than the microcontroller pins can safely provide. Instead, the Arduino only sends low-power control signals to the DRV8871 input pins, while the motor drivers switch the higher-current motor supply to the motors. Each driver has two Arduino control inputs and two motor output terminals. By applying PWM to one input while holding the other low, the motor can be driven in one direction. Reversing which input receives the PWM signal reverses the motor direction. The schematic for the motor driver is shown below:

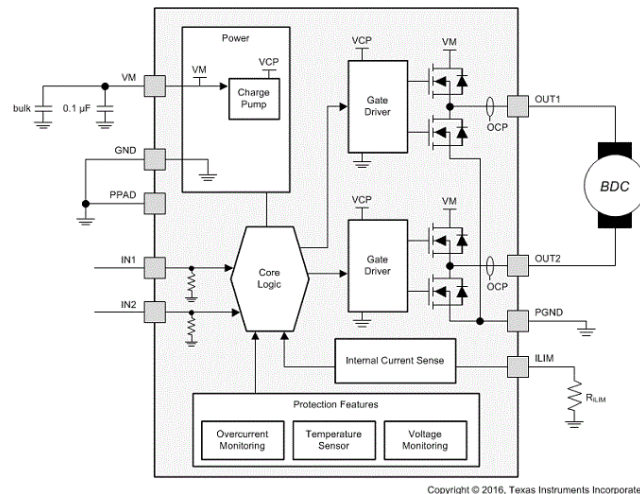


Figure 5: DRV8871 Motor Driver schematic

The robot was powered using an onboard 12 V battery pack so that the system could operate independently during balancing and Bluetooth-controlled movement. The battery pack supplied power to the main electrical system, including the motor drivers, motors, and Arduino controller. Using a battery pack instead of a wired bench supply was necessary for the final design because the robot needed to move freely across the floor, perform forward and backward motion, turn, and operate on a ramp without being restricted by external power cables.

Power distribution was separated into two main parts: motor power and control power. The motors required the highest current in the system, so the battery supply was connected to the VM and GND power terminals of the DRV8871 motor drivers. Each DRV8871 then switched this motor supply to its corresponding DC gearmotor based on the PWM and direction signals received from the Arduino. This allowed the motors to draw the current they needed from the battery pack rather than from the Arduino, which would not be able to safely supply the motor current directly.

The Arduino was also powered from the robot's battery system. The Arduino board contains onboard voltage regulation circuitry, including a step-down regulator, which allows the board to generate the regulated voltage needed for its internal electronics. Although the robot used a higher-voltage battery

supply for the motors, the Arduino itself operates using 3.3 V logic. This means that its input and output pins are designed for 3.3 V signal levels, not 5 V or higher.

Because the Arduino uses 3.3 V logic, special care was required when connecting the motor encoders. The Pololu gearmotor encoders have separate power, ground, channel A, and channel B wires. The encoder output signals switch between ground and the encoder supply voltage. The encoder operating voltage is between 3.5 and 20V, so it cannot be supplied with any voltage that is also safe for the arduino. To protect the Arduino input pins, voltage dividers were considered for the encoder signal lines.

A voltage divider reduces the voltage of a signal before it reaches the Arduino. For each encoder output, the encoder signal would be connected through two resistors arranged as a divider, with the Arduino input connected at the midpoint. This scales the encoder pulse signal down to a safe level below 3.3 V while still allowing the Arduino to detect the digital pulses. Since each motor has two encoder outputs, channel A and channel B, four voltage dividers would be required in total: left encoder A, left encoder B, right encoder A, and right encoder B. Our calculations for the voltage divider resistances are shown in Appendix A.

The schematic for the gearmotor with encoders are shown in the following figure:

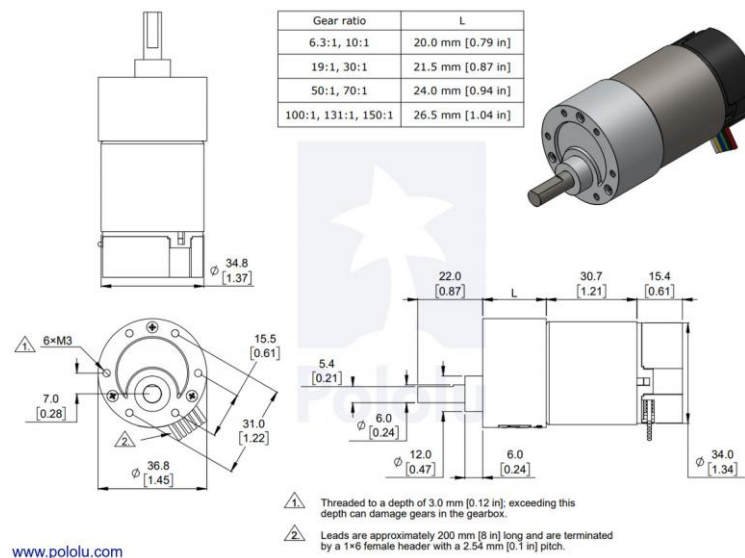


Figure 6: Dimensions of the 37D metal gearmotors with 64 CPR encoders

Lastly, a common ground was required throughout the entire circuit. The battery ground, Arduino ground, DRV8871 grounds, and encoder grounds were connected together so that all signal voltages shared the same reference point. This was important because the Arduino needed to correctly interpret the PWM, direction, and encoder signals. Without a common ground, the logic signals between the Arduino, motor drivers, and encoders could become unreliable or unreadable.

The final wiring layout therefore combined three main subsystems. The sensing subsystem consisted of the onboard IMU and the two motor encoders. The control subsystem consisted of the Arduino Nano running the balance, velocity, and Bluetooth control code. The actuation subsystem consisted of the two DRV8871 motor drivers and the left and right DC gearmotors. Together, these subsystems

allowed the robot to estimate its tilt angle, calculate the required correction, and drive the wheels in the correct direction to maintain balance.

Software and Computer Design:

The final robot software was organized into several main subsystems: sensor initialization, IMU processing, encoder processing, control loops, BLE communication, and motion state logic. The Arduino Nano acted as the central controller. It read the onboard IMU, processed encoder counts from both motors, received commands over BLE, calculated the required control output, and sent PWM commands to the left and right DRV8871 motor drivers.

The software was structured so that low-level functions handled repeated hardware tasks, while the main loop combined these functions into the full balancing system. For example, the motor output functions `setleftmotor()` and `setrightmotor()` handled the conversion from signed PWM commands to H-bridge signals, while the encoder interrupt functions continuously updated the left and right encoder counts. The main loop then used these sensor values to compute the filtered angle, position error, target angle, and final PWM output.

For sensor processing and filtering, the robot used the onboard IMU on the Arduino Nano to estimate the tilt angle of the robot body. The accelerometer provided an absolute angle reference based on gravity, while the gyroscope provided the angular velocity of the robot. Since the accelerometer can be noisy during motion and the gyroscope can drift over time, the final code combined both measurements using a complementary filter.

At startup, the robot first calibrated the gyroscope while the robot was kept still. In the final code, this was done by collecting 500 gyroscope samples and averaging them to estimate the gyroscope bias. The x-axis gyroscope bias was stored as `gyroBiasX` and later subtracted from all future gyroscope readings:

```
void calibrateGyroX() {
    const int samples = 500;
    float sum = 0.0;
    float gx, gy, gz;

    Serial.println("Keep robot still. Calibrating gyro...");

    for (int i = 0; i < samples; i++) {
        while (!IMU.gyroscopeAvailable()) {
        }
        IMU.readGyroscope(gx, gy, gz);
        sum += gx;
        delay(5);
    }

    gyroBiasX = sum / samples;
    gyroBiasZ = sumZ / samples;
}
```

This calibration step was important because even a small constant gyro offset can accumulate into a large angle error when integrated over time. By measuring the bias at startup and subtracting it during operation, the software reduced long-term drift in the angle estimate.

After calibration, the accelerometer was used to initialize the starting angle of the filter. This gave the complementary filter a real physical reference angle before the main balancing loop began:

```
void initAccel() {
    IMU.readAcceleration(x, y, z);
    accel_angle = atan2(y, z) * 180.0 / PI;
    filtered_angle = accel_angle;

    angleOffset = 0; //-0.955
}
```

During the main loop, the Arduino read the accelerometer and calculated the accelerometer-based angle using the measured (y) and (z) acceleration values:

```
IMU.readAcceleration(x, y, z);
accel_angle = atan2(y, z) * 180.0 / PI + angleOffset;
```

The gyroscope reading was then corrected by subtracting the measured bias. The sign of the gyroscope reading was also reversed so that the gyro direction matched the sign convention used by the controller:

```
IMU.readGyroscope(gyro_x, gyro_y, gyro_z);
    gyro_x -= gyroBiasX; // Remove bias
    gyro_x = -gyro_x; //fix wrong direction of acceleration
```

The loop time was calculated using `micros()` instead of assuming a fixed sample period. This made the filter more accurate because the loop timing can vary slightly due to sensor reads, BLE processing, serial printing, and control calculations:

```
unsigned long current_time = micros();
    dt = (current_time - last_time) / 1000000.0;
    last_time = current_time;
```

The final filtered angle was then calculated using the complementary filter:

```
float k = 0.98;
filtered_angle = k*(filtered_angle + gyro_x* dt) + (1-k)*accel_angle;
```

After completing the sensor processing and filtering, the next thing we worked on was the control algorithms. The final control algorithm used a double cascade control structure. The outer loop controlled the robot's wheel position using encoder feedback, while the inner loop controlled the robot's body angle using the filtered IMU angle. The purpose of this structure was to allow the robot

to move to a target position while still maintaining fast balance correction. Instead of directly converting a position error into motor PWM, the position loop generated a target lean angle. The angle loop then generated the motor PWM needed to reach that lean angle.

The inner loop was the main balancing loop. It compared the measured filtered angle with the target angle generated by the position controller. In the code, the angle error was calculated as:

```
angleError = filtered_angle - targetAngle;
```

The derivative term was taken directly from the gyroscope reading:

```
derivative = gyro_x;
```

Using the gyroscope as the derivative term was useful because the gyroscope already measures angular velocity. This avoided having to numerically differentiate the filtered angle, which could amplify noise. The integral term accumulated the angle error over time:

```
integral += angleError * dt;  
integral = constrain(integral, -(PWM_MAX / KI), PWM_MAX / KI);
```

The integral clamp was included to prevent the integral term from growing too large. This is important for a self-balancing robot because excessive accumulated error can cause the robot to overcorrect and oscillate. After calculating the proportional, derivative, and integral terms, the angle PID controller produced the base motor PWM command:

```
basePWM = CONTROL_SIGN * ((KD * derivative) + (KP * angleError) + (KI *  
integral));  
basePWM = constrain(basePWM, -PWM_MAX, PWM_MAX);
```

The constant, CONTROL_SIGN, was used to make sure that the motor correction direction matched the physical direction of the robot. If the robot tilted forward, the wheels needed to drive forward; if the robot tilted backward, the wheels needed to drive backward. The output was also constrained between -255 and 255, matching the valid PWM range used by the motor functions. The final angle controller gains were:

```
const float KP = 8.25;    // 7.75  
const float KD = 0.5;     // 0.5  
const float KI = 45;     // 50
```

The proportional gain controlled how strongly the robot responded to angle error. The derivative gain provided damping by responding to angular velocity. The integral gain helped correct long-term bias, such as a small offset in the balance angle or center of mass. These values were tuned experimentally to keep the robot upright without excessive oscillation.

The outer loop was the position control loop. This loop ran more slowly than the angle loop and was updated every 40 ms. Inside the position loop, the code first updated the encoder values and calculated the current robot position by averaging the left and right encoder counts:

```
const unsigned long PC_LOOP_DELAY = 40;
computeEncoderValues();
currentPosition = ((float)LEncCount + (float)REncCount) / 2.0;
```

Averaging the two encoder counts gave one approximate position value for the robot. The position error was then calculated by comparing the current position with the smoothed target position:

```
positionError = currentPosition - targetPositionSmoothed;
```

The code also used target smoothing to prevent sudden jumps in the commanded position. Instead of immediately changing targetPositionSmoothed to the new target, the code limited how quickly the target could change:

```
float targetDelta = targetPosition - targetPositionSmoothed;
float maxStep = TARGET_SLEW_RATE * PC_dt;
if (fabs(targetDelta) < maxStep) {
    targetPositionSmoothed = targetPosition;
} else {
    targetPositionSmoothed += (targetDelta > 0 ? maxStep : -
maxStep);
}
```

This was important because sudden changes in target position would create a large position error, causing the robot to lean too aggressively. By smoothing the target position, the robot accelerated and decelerated more gradually.

The position derivative term was calculated using the measured change in position, with the expected target velocity subtracted:

```
float targetVelocity = (targetPositionSmoothed - lastTargetSmoothed) /
PC_dt;
//Serial.println(targetVelocity);
lastTargetSmoothed = targetPositionSmoothed;

positionDerivative = ((currentPosition - lastPosition) / PC_dt) -
targetVelocity;
```

This made the derivative term respond to the difference between the robot's actual motion and the desired motion of the smoothed target. The position controller then calculated the target angle:

```
targetAngle = ((PC_KP * positionError) + (PC_KD * positionDerivative) +
(PC_KI * positionIntegral));
```

The target angle was then constrained to prevent the position loop from commanding an unsafe lean angle:

```
targetAngle = constrain(targetAngle, -MAX_ANGLE, MAX_ANGLE);
```

For normal flat-ground movement, MAX_ANGLE was set to 3 degrees. This limited the maximum lean angle commanded by the position loop, helping the robot move without tipping too aggressively. The final position control gains were:

```
float PC_KP = 0.0015; // 0.001 ; 0.0015
float PC_KD = 0.00075; // 0.00125; 0.002
float PC_KI = 0.0003; //0.0003 ;
```

The position integral was also limited to prevent the outer loop from building up too much accumulated error:

```
positionIntegral = constrain(positionIntegral, -
((integralWeight*MAX_ANGLE) / PC_KI), ((integralWeight*MAX_ANGLE) /
PC_KI));
```

The code only added to the position integral when the calculated target angle was not saturated:

```
if (fabs(targetAngle) < MAX_ANGLE && ((fabs(PC_KI*positionIntegral)) <
integralWeight*MAX_ANGLE *0.95)) {
    positionIntegral += positionError * PC_dt;
}
```

This helped reduce integral windup in the position loop. If the target angle was already at the maximum allowed value, continuing to accumulate more position error would not make the robot move faster, but it could cause overshoot later.

After the base PWM was calculated by the angle loop, the final motor commands were generated. A heading correction term was added to one motor and subtracted from the other motor so that the robot could correct its heading or perform turns:

```
int leftPWM = applyDeadband(basePWM + headingCorrection, L_DEADBAND);
int rightPWM = applyDeadband((basePWM -
headingCorrection)*imbalanceMultiplier, R_DEADBAND);
```

The applyDeadband() function compensated for the minimum PWM needed to overcome motor friction:

```
int applyDeadband(int pwm, int deadband) {
```

```

if (pwm > 0) {
    return constrain(pwm + deadband, 0, PWM_MAX);
}
else if (pwm < 0) {
    return constrain(pwm - deadband, -PWM_MAX, 0);
}
else {
    return 0;
}
}

```

This was necessary because the motors did not begin rotating at very small PWM values. Without deadband compensation, small control outputs from the PID controller might not move the wheels, causing the robot to respond too slowly near equilibrium.

Finally, the corrected PWM values were sent to the left and right motors:

```

setLeftMotor(leftPWM);
setRightMotor(rightPWM);

```

The last major thing for software was BLE Control. Bluetooth Low Energy was used to send movement commands from an Android phone to the robot. The BLE phone app was based on a template provided by the teaching team. Therefore, the main software task was not to build the app from scratch, but to understand how the template transmitted button commands and then modify the Arduino-side command handling so that each button triggered the correct robot behaviour.

In the final code, the Arduino was configured as the BLE peripheral device. A custom BLE service and characteristic were created so that the phone app could connect to the robot and write command values:

```

BLEService customService("00000000-5EC4-4083-81CD-A10B8D5CF6EC");
BLECharacteristic customCharacteristic(
    "00000001-5EC4-4083-81CD-A10B8D5CF6EC", BLERead | BLEWrite |
    BLENotify, BUFFER_SIZE, false);

```

During setup, the Arduino initialized BLE, set the advertised device name, added the custom service, and began advertising. This allowed the Android phone to detect and connect to the robot:

```

BLE.setLocalName("BLE-DEVICEG1");
BLE.setDeviceName("BLE-DEVICEG1");

// Add the characteristic to the service
customService.addCharacteristic(customCharacteristic);

// Add the service
BLE.addService(customService);

```

```

// Set an initial value for the characteristic
customCharacteristic.writeValue("Waiting for data");

// Start advertising the service
BLE.advertise();

```

Once connected, the phone app sent text-based commands to the BLE characteristic when the user pressed buttons in the app. The Arduino checked whether a new command had been written, copied the received data into a string, and compared the string with the expected command names:

```

if (customCharacteristic.written()) {
    // Get the length of the received data
    int length = customCharacteristic.valueLength();

    // Read the received data
    const unsigned char* receivedData =
customCharacteristic.value();

    // Create a properly terminated string
    char receivedString[length + 1]; // +1 for null terminator
    memcpy(receivedString, receivedData, length);
    receivedString[length] = '\0'; // Null-terminate the string
    Serial.print(receivedString);

    if(strcmp(receivedString, "FORWARD")==0){
        currentCommand = 0;
    }
    else if(strcmp(receivedString, "BACK")==0){
        currentCommand = 1;
    }
    else if (receivedString[0] =='B'){ //stop
        currentCommand = 2;
    }
    else if (strcmp(receivedString, "LEFT")==0){
        currentCommand = 3; //turn left 45
    }
    else if (strcmp(receivedString, "RIGHT")==0){
        currentCommand = 4; //turn right 45
    } else if (receivedString[0] == 'A') {
        currentCommand = 5; // Ramp function
    } else if (receivedString[0] == 'C') {
        currentCommand = 6; // RFU (Reserved for future use)
    }
}

```

```

    }

    // Optionally, respond by updating the characteristic's value
    customCharacteristic.writeValue("Data received");
}

```

The BLE commands were treated as high-level movement requests rather than direct motor commands. For example, the forward and backward buttons changed the target position, the left and right buttons triggered the turning logic, and the ramp button triggered the ramp sequence. This was important because the motors still needed to be controlled through the balancing controller. If the BLE app directly controlled PWM, the robot could become unstable. By using BLE only to update `currentCommand`, the final code kept the balance controller in charge of the motor outputs at all times.

Design Evolution and Engineering Decisions

Between the midpoint demo and the final demo, the mechanical and electrical design of the robot remained mostly unchanged. The main design evolution occurred in the software. The midpoint demo code was mainly focused on achieving basic self-balancing and simple BLE-controlled movement. After the midpoint demo, the software was expanded to support the full final demo requirements: stationary balancing, straight-line movement, 45° and 90° turns, and ramp traversal. The following sections compare the midpoint and final demo code and explain the major engineering decisions made during this process.

Velocity Control to Encoder Position Control

A major software change after the midpoint demo was replacing the midpoint velocity loop with an encoder-based position loop. In the midpoint code, the outer control loop was designed around wheel speed. The encoder counts were used to calculate the left and right wheel RPM, and the average RPM was compared with a target RPM. This allowed the robot to move forward or backward, but it did not directly control how far the robot travelled.

In the midpoint code, the velocity loop gains were defined as:

```

const float VKP = 0.03;
const float VKI = 0.05;
const float VKD = 0.03;

```

The robot calculated its average wheel speed by averaging the left and right motor RPM values:

```

computeEncoderValues();
avgRPM = (Lrpm+Rrpm)/2.0;

```

Since encoder-based speed measurements can be noisy, the midpoint code also applied a simple low-pass filter to the measured RPM:

```
filteredRPM = 0.8*filteredRPM+(1-0.8)*avgRPM; //this is like a low pass
filter so the heavy noise is managed
```

The filtered RPM was then compared with the target RPM:

```
RPMErrror = filteredRPM-targetRPM;
float Vderivative = (RPMErrror-lastRpmError)/velocity_DT;
Vintegral = Vintegral + RPMErrror*velocity_DT;
lastRpmError=RPMErrror;
```

Finally, the velocity controller adjusted TARGET_ANGLE, which acted as the lean angle command for the balance controller:

```
if(abs(RPMErrror)>RPMDeadZone){
    TARGET_ANGLE = (VKP*RPMErrror) +
(VKI*Vintegral)+(VKD*Vderivative);
}
TARGET_ANGLE = constrain((TARGET_ANGLE),-5,5);
}
```

This approach was useful for the midpoint demo because it allowed the robot to move by leaning forward or backward until the wheel speed approached the target RPM. This approach was useful for the midpoint demo because it allowed the robot to move by leaning forward or backward until the wheel speed approached the target RPM. The BLE commands in the midpoint code also matched this structure. For example, the forward and backward commands simply changed the target RPM:

For the final demo, the outer loop was changed into a position controller. Instead of using RPM as the main feedback variable, the final code used the average of the left and right encoder counts to estimate the robot's current position:

```
computeEncoderValues();
currentPosition = ((float)LEncCount + (float)REncCount) / 2.0;
```

The final code then compared this measured position with a target position:

```
positionError = currentPosition - targetPositionSmoothed;
```

This was a more appropriate structure for the final demo because the desired forward and backward movements could now be represented as encoder count targets. For example, the final code used the BLE forward command to set a target position of 3750 counts, while the backward command returned the target position to zero:

```
switch (currentCommand) {
    case 0: // forward
        MAX_ANGLE = 3.0;
        homeSet = false;
        targetPosition = 3750;
```

```

        targetHeading = 0;
        break;
    case 1:          // backwards
        MAX_ANGLE = 3.0;
        homeSet = false;
        targetPosition = 0;
        //NOTE: If it is permissible to hit STOP between forward and
back tests, this is acceptable. If not,
        // change targetPosition = 0. Seek prof/TA guidance
        targetHeading = 0;
        break;

```

The position controller then converted the position error into a target lean angle:

```

targetAngle = ((PC_KP * positionError) + (PC_KD * positionDerivative) +
(PC_KI * positionIntegral));
targetAngle = constrain(targetAngle, -MAX_ANGLE, MAX_ANGLE);

```

The final position loop gains were:

```

float PC_KP = 0.0015; // 0.001 ; 0.0015
float PC_KD = 0.00075; // 0.00125; 0.002
float PC_KI = 0.0003; //0.0003 ;

```

This made the final controller a cascade system. The outer position loop did not directly control motor PWM. Instead, it calculated the lean angle needed to move toward the target position. The inner balance PID loop then used this targetAngle to generate the motor PWM needed to keep the robot balanced while moving. This was better than directly commanding motor speed because it preserved the balancing behaviour at all times.

Another change in the final code was that the position loop ran more frequently than the old velocity loop. The midpoint velocity loop used a delay of 100 ms:

```

const float VELOCITY_LOOP_DELAY=100; //in milliseconds

```

In the final code, the position loop updated every 40 ms:

```

const unsigned long PC_LOOP_DELAY = 40;

```

This gave the final controller more frequent position updates, which helped the robot respond more smoothly to changes in position error. At the same time, the position loop still ran slower than the main angle loop, which was important because the angle loop needed to remain the fastest part of the controller for stable balancing.

The final code also added target smoothing before calculating the position error:

```
float targetDelta = targetPosition - targetPositionSmoothed;
float maxStep = TARGET_SLEW_RATE * PC_dt;

if (fabs(targetDelta) < maxStep) {
    targetPositionSmoothed = targetPosition;
} else {
    targetPositionSmoothed += (targetDelta > 0 ? maxStep : -
maxStep);
}
```

This prevented the target position from changing instantly when a command was received. Without this, the position error could suddenly become very large, causing the outer loop to command a large lean angle. By limiting how quickly `targetPositionSmoothed` could change, the final code made the robot accelerate more gradually and reduced the chance of sudden lurching or oscillation.

Several alternatives were considered before using position control. One option was to keep the midpoint velocity loop and control distance by running the robot forward for a fixed amount of time. This would have been simpler, but we believed that it would not be as reliable because the same time duration could produce different distances depending on the battery level, motor response, or floor friction. Another option was to directly command a fixed PWM value for forward and backward motion, but this would bypass the feedback loop and make the robot much less consistent. Encoder-based position control was chosen because it used measured wheel motion to determine how far the robot had actually travelled.

Overall, changing from velocity control to position control made the final robot more suitable for the final demo requirements. The midpoint velocity loop was useful for early movement testing, but the final position loop gave the robot a measurable destination and allowed it to move toward that destination while staying balanced. The main lesson from this change was that controlling speed is not the same as controlling position. For a self-balancing robot that must move a specific distance and stop accurately, encoder position feedback is more useful.

Straight-Line Movement and Target Smoothing

After implementing the position loop, straight-line movement was also changed from a speed-commanded behaviour into a target-position behaviour with smoothing. In the midpoint code, pressing the forward or backward button changed the desired wheel speed by setting `targetRPM`. For example, the forward command set the target speed to 16 RPM, while the backward command set it to -16 RPM:

```
if(currentCommand == 0){
    targetRPM = 16;
    turning_value = 0;
}
else if (currentCommand == 1){
```

```

    targetRPM = -16;
    turning_value = 0;
}
else if (currentCommand == 2){
    targetRPM = 0;
    turning_value = 0;
}

```

This approach was useful for early testing because it allowed the robot to move forward and backward while still balancing. However, it did not give the robot a fixed destination. The robot would continue trying to reach the commanded RPM until a new command was sent, so the final travel distance depended on how long the command was applied, how quickly the robot accelerated, the battery level, the floor surface, and the amount of motor slip. This made it difficult to consistently move a required distance and stop accurately.

In the final demo code, the forward and backward commands were changed so that they updated targetPosition instead of targetRPM. The forward command set the target encoder position to 3750 counts, while the backward command set the target back to zero:

```

switch (currentCommand) {
    case 0:          // forward
        MAX_ANGLE = 3.0;
        homeSet = false;
        targetPosition = 3750;
        targetHeading = 0;
        break;
    case 1:          // backwards
        MAX_ANGLE = 3.0;
        homeSet = false;
        targetPosition = 0;
}

```

This made the straight-line movement more repeatable because the robot was no longer just trying to maintain a speed. Instead, it was trying to move toward a measured encoder position. The encoder counts provided feedback on how far the wheels had actually rotated, allowing the controller to reduce the commanded lean angle as the robot approached the target.

The final code also improved the stop command. In the midpoint version, stopping only set targetRPM to zero:

```

else if (currentCommand == 2){
    targetRPM = 0;
    turning_value = 0;
}

```

In the final version, the stop command reset the target position, target heading, smoothed target, and encoder home position:

```
case 2:          // stop
    MAX_ANGLE = 3.0;
    targetPosition = 0;
    targetHeading = 0;
    targetPositionSmoothed = 0;
    lastTargetSmoothed = 0;
    if (!homeSet) {
        positionError = 0;
        currentPosition = 0;
        REncCount = 0;
        LEncCount = 0;
        homeSet = true;
    }
    break;
}
```

This made the stop state more useful for the final demo because it gave the robot a known reference position before the next movement command. Resetting the encoder counts also helped the forward and backward commands behave more consistently, since the forward movement could start from a known zero position.

A major issue with position commands is that an instant change in target position can create a large position error. For example, if the target position immediately jumps from 0 counts to 3750 counts, the position controller sees a large error right away. This can cause the outer loop to command a large target angle, making the robot lurch forward and possibly destabilize. To reduce this effect, the final code added target smoothing using `targetPositionSmoothed` and `TARGET_SLEW_RATE`:

```
float targetPositionSmoothed = 0;
float lastTargetSmoothed = 0;
float TARGET_SLEW_RATE = 900.0; // counts per second
```

Instead of using `targetPosition` directly in the position error calculation, the code first limited how quickly `targetPositionSmoothed` could move toward `targetPosition`:

```
float targetDelta = targetPosition - targetPositionSmoothed;
float maxStep = TARGET_SLEW_RATE * PC_dt;
if (fabs(targetDelta) < maxStep) {
    targetPositionSmoothed = targetPosition;
} else {
    targetPositionSmoothed += (targetDelta > 0 ? maxStep : -maxStep);
}
```

This works like a ramped command input. If the final target is far away, the smoothed target only changes by a maximum amount each loop. As a result, the commanded position changes gradually instead of jumping instantly. The position controller then calculates the error using the smoothed target:

```
positionError = currentPosition - targetPositionSmoothed;
```

This made the straight-line motion smoother because the robot leaned into the movement more gradually. It also reduced the chance that the position loop would command the maximum allowed lean angle immediately after a button press.

The final code also calculated the velocity of the smoothed target and subtracted it from the measured position derivative:

```
float targetVelocity = (targetPositionSmoothed - lastTargetSmoothed) /  
PC_dt;  
    //Serial.println(targetVelocity)  
lastTargetSmoothed = targetPositionSmoothed;  
positionDerivative = ((currentPosition - lastPosition) / PC_dt) -  
targetVelocity;
```

This was important because the target itself was moving during the smoothing process. Without accounting for the target velocity, the derivative term would respond as if the robot were simply lagging behind a moving target. By subtracting targetVelocity, the derivative term better represented the difference between the robot's actual motion and the desired motion of the smoothed target.

Straight-line movement also required the robot to maintain its heading while travelling. In the midpoint code, the forward and backward commands set turning_value to zero, so both motors received the same balance command apart from motor differences:

```
Turning_value = 0;
```

In the final code, straight-line movement also reset the desired heading to zero:

```
targetHeading = 0;
```

This allowed the final heading correction logic to help the robot keep a straighter path. The motor commands were calculated by adding the heading correction to one side and subtracting it from the other:

```
int leftPWM = applyDeadband(basePWM + headingCorrection,  
L_DEADBAND);  
int rightPWM = applyDeadband((basePWM -  
headingCorrection)*imbalanceMultiplier, R_DEADBAND);
```

Although the full turning and heading-control logic will be discussed in the turning section, its use during straight-line movement was important because it helped reduce drift. If the robot began rotating slightly while driving forward or backward, the heading correction could apply a small differential motor command to bring it back toward the target heading.

Several alternatives were considered before using this final structure. The simplest option was to keep the midpoint approach and control straight-line movement using only targetRPM. This was easier to implement, but it could not guarantee a specific travel distance. Another option was to use a fixed PWM value for a fixed amount of time, but that would be even more sensitive to battery voltage, motor mismatch, and floor friction. A third option was to use position control without target smoothing, but that caused more sudden target-angle changes and made the robot more likely to lurch. The final approach combined encoder-based position targets with target smoothing because it gave the robot both a measurable destination and a smoother transition toward that destination.

Overall, the straight-line movement improved because the final code controlled where the robot should go, not just how fast it should move. The midpoint code was useful for proving that the robot could move while balancing, but the final code was better suited for the final demo because it used encoder position targets, reset the reference position during stop, limited sudden target changes, and included heading correction for straighter travel. The main lesson was that smooth command generation is just as important as the controller gains. Even a stable controller can become unstable if the target changes too suddenly.

Turning Control:

Turning control was one of the largest software changes between the midpoint demo and the final demo. In the midpoint code, turning was controlled using a simple open-loop PWM offset. When the robot received a turn command, the code set targetRPM to zero and applied a constant turning_value to the left and right motor commands:

```
else if (currentCommand == 3){
    targetRPM = 0;
    turning_value = 8;
}
else if (currentCommand == 4){
    targetRPM = 0;
    turning_value = -8;
}
```

This turning value was then subtracted from one motor and added to the other:

```
int leftPWM = applyDeadband(basePWM, L_DEADBAND)-turning_value;
int rightPWM = applyDeadband(basePWM, R_DEADBAND)+turning_value;
```

This approach was useful for the midpoint demo because it was simple and allowed the robot to rotate while still using the balance controller. However, it was open-loop. The robot had no measurement of how far it had turned, so the final heading depended on battery voltage, friction, motor mismatch, and

how long the command was held. A fixed PWM offset could make the robot turn, but it could not reliably produce a specific 45° or 90° rotation.

For the final demo, turning was changed into a feedback-based heading control system. The final code added a heading estimate called `zAngle`, which was calculated by integrating the z-axis gyroscope reading over time. First, the code read the gyroscope values and was corrected by a measured offset that we found through excel calculations.

```
IMU.readGyroscope(gyro_x, gyro_y, gyro_z);  
gyro_z = gyro_z - 0.149990639;
```

After the loop time `dt` was calculated, the z-axis angular velocity was integrated to update the heading estimate:

```
zAngle += gyro_z * dt;
```

This was an important improvement because the robot could now estimate how much it had rotated about the vertical axis. Instead of only applying a fixed turn offset, the final code compared the measured heading `zAngle` with a desired heading `targetHeading`. The final code used this comparison to generate a heading correction:

```
if (zAngle > targetHeading + 2) { //adding margin to prevent  
oscillation  
    if (turning) {  
        headingCorrection = 8;  
    } else {  
        headingCorrection = 3;  
    }  
} else if (zAngle < targetHeading - 2) {  
    if (turning) {  
        headingCorrection = -8;  
    } else {  
        headingCorrection = -3;  
    }  
} else {  
    headingCorrection = 0;  
}  
}
```

The $\pm 2^\circ$ range acted as a tolerance band. Without this tolerance, the robot could constantly switch correction direction when it was very close to the target heading, causing unnecessary oscillation. The code also used two correction sizes. During an active turn, `headingCorrection` was set to ± 8 to rotate the robot more strongly. During normal motion, the correction was reduced to ± 3 so that small heading errors could be corrected without causing aggressive turning.

The heading correction was applied to the motor outputs by adding it to one side and subtracting it from the other:

```
int leftPWM = applyDeadband(basePWM + headingCorrection, L_DEADBAND);
int rightPWM = applyDeadband((basePWM -
headingCorrection)*imbalanceMultiplier, R_DEADBAND);
```

This created a differential drive effect. If the robot needed to rotate, one motor received a slightly larger command while the other received a smaller command. At the same time, both motors still included the main basePWM from the balance controller, so the robot continued balancing while it turned. The final code also included an imbalanceMultiplier on the right motor command. This was not part of the main turning strategy itself, but was included as a practical correction for motor mismatch between the two wheels. The reasoning and tuning process for this multiplier, along with the motor deadband values, is discussed under Challenge 2 in the Challenges and troubleshooting section.

The final code also added a turn state machine to perform the required staged turns. The midpoint code only applied a turning offset when a turn command was active, but the final code organized turning into states:

```
enum turnStates {
    turnIdle, add45, turnWait
};
```

When a left or right command was received, the code set the turn direction and started the state machine:

```
case 3: //left turn
    MAX_ANGLE = 3.0;
    turnDirection = 1; // set the turn direction to be modified
by state machine
    turnState = add45; // start turn sequences
    break;
case 4: // right turn. same implementation as above but with opposite
direction
    MAX_ANGLE = 3.0;
    turnDirection = -1;
    turnState = add45;
    break;
```

The add45 state changed the target heading by 45° in the selected direction:

```
case add45:
    targetHeading += 45*turnDirection;
    imbalanceMultiplier = 1.07;
    turnState = turnWait;
    turnStartTime = millis();
    break;
```

After commanding a 45° heading change, the robot entered turnWait. This gave the robot time to rotate and settle before adding the next 45° step:

```
case turnWait:
    if (millis() - turnStartTime >= TURN_WAIT_TIME) {
        if (fmod(targetHeading, 90.0) == 0) {
            turnState = turnIdle;
            currentCommand = 8; //dummy command for init reasons
        } else turnState = add45;
        turnStartTime = millis();
    }
    break;
```

This structure was useful because the final demo required the robot to turn in two 45° stages, with a pause between each stage. Instead of trying to complete the full 90° turn in one large motion, the final code incremented the heading target by 45°, waited, and then added the second 45° step. This made the turn easier to control and gave the robot time to regain stability between turning motions.

Several alternatives were considered before using this final turning structure. The simplest option was to keep the midpoint open-loop method and tune turning_value or turn duration manually. This would have been easier to implement, but it would not be reliable because the same PWM offset could produce different turn angles under different conditions. Another option was to estimate heading from the difference between left and right encoder counts. However, encoder-based turning can be affected by wheel slip, especially because the robot is constantly making small balancing corrections. The z-axis gyroscope was chosen because it directly measures the robot's rotational velocity, making it more suitable for estimating heading during turns.

Overall, the final turning system was a clear improvement over the midpoint design because it changed turning from an open-loop motor offset into a feedback-based heading correction system. The midpoint code could make the robot rotate, but the final code could estimate heading, compare it with a target, and correct until the robot reached the desired angle. The main lesson was that turning a self-balancing robot requires both balance control and heading feedback. The robot cannot simply rotate using fixed PWM values because it must remain upright while also reaching a specific final heading.

Ramp Development:

Ramp traversal was added later in the project, after the midpoint demo. The midpoint demo code did not include any ramp-specific behaviour. It only included balancing, simple forward and backward movement, and open-loop turning. Since the final demo required the robot to move up, remain stationary on, and move down a ramp while maintaining balance, a new ramp-control mode had to be developed after the midpoint stage.

Because ramp development began late, the team decided to split the work between two different coders. Instead of immediately integrating the ramp behaviour into the main final-demo code, separate

ramp-specific versions were created for testing different ramp angles. This resulted in separate files for the 5°, 10°, and 15° ramp tests. This was a practical engineering decision because it allowed ramp tuning to be performed independently without risking changes to the main balancing, straight-line, and turning code.

The main final code included a general ramp structure using a finite state machine. When the ramp command was received, the code entered rampClimb and began the ramp sequence:

```
case 5: //ramp
    rampState = rampClimb;
    rampStartTime = millis();
    break;
}
```

The main final code also defined several ramp states:

```
enum rampStates {
    rampIdle, rampClimb, rampClimb10, rampClimb15, rampWait, rampDescend,
    rampComplete
};
```

This structure showed the intended approach for ramp traversal: climb the ramp, wait at the top, descend, and then return to the idle state. However, the separate ramp files were used to tune the actual values for different ramp angles. These files kept the same general control structure, but changed important parameters such as MAX_ANGLE, TARGET_SLEW_RATE, targetPosition, wait time, and integral limiting.

For the 5° ramp file, the ramp controller increased the allowable lean angle and commanded a target position of 3900 encoder counts during the climb:

```
case rampClimb:
    TARGET_SLEW_RATE = 1700.0;
    targetPosition = 3900;
    integralWeight = 0.5;
    MAX_ANGLE = 16.0;

    PC_KI = 0.0003;
    integralCLAMPER = 21000;

    if ((LEncCount+REncCount/2.0) > 3900) {
        rampState = rampWait; //REVERT
        rampStartTime = millis();
    }
    break;
```

This was different from normal flat-ground movement, where the maximum lean angle was much smaller. On the ramp, the robot needed more authority to lean forward and climb against gravity, so MAX_ANGLE was increased. The target slew rate was also increased so that the target position could advance more quickly during the climb.

For the 10° ramp file, the ramp controller used a larger target distance and introduced an intermediate rampClimb10 stage:

```
case rampClimb10:
    integralCLAMPER = 26700;
    //26300; that goes to middle of 10 degrees

    if((LEncCount+REncCount/2.0) > 7400){
        rampStartTime = millis();
        rampState = rampWait;
    }

    if((LEncCount+REncCount/2.0) < 4000){
        rampState = rampClimb;
    }

    break;
```

This version shows how the ramp control evolved as the ramp became steeper. The robot needed a larger travel target and more staged logic to continue climbing reliably. The use of encoder-count thresholds helped the code decide when the robot had reached different parts of the ramp instead of relying only on time.

For the 15° ramp file, the controller was adjusted again to provide even more climbing authority. The idle and climb states used a larger maximum angle of 17°:

```
case rampIdle:
    MAX_ANGLE = 17.0;
    TARGET_SLEW_RATE = 900.0;
    PC_KP = 0.001; // 0.001 ; 0.0015
    PC_KD = 0.00125; // 0.00125; 0.002
    PC_KI = 0.0003; //0.0003 ;
    integralWeight = 1.0;
    //do nothing and restore stable max angle. Return after complete
    break;
    case rampClimb:
        TARGET_SLEW_RATE = 1800.0;
        targetPosition = 7900;
        integralWeight = 0.5;
        MAX_ANGLE = 17.0;
```

```

PC_KI = 0.0003;
integralCLAMPER = 20000;

if ((LEncCount+REncCount/2.0) > 3900) {
    rampState = rampClimb10; //REVERT
    rampStartTime = millis();
}
break;

```

The 15° version also used a larger integral clamp during the steepest part of the climb:

```

case rampClimb15:
    integralCLAMPER = 47500; //was 31650
    if((LEncCount+REncCount/2.0) > 8000){
        rampState = rampWait;
    }

    if((LEncCount+REncCount/2.0) < 7300){
        rampState = rampClimb15; //keep it the same it needs that
boost
    }

    break;

```

This change was made because the steeper ramp required more sustained correction from the controller. Compared with flat-ground balancing, ramp climbing creates a larger steady disturbance due to gravity pulling the robot downhill. Allowing more accumulated correction helped the robot continue climbing instead of stalling or rolling backward.

The descent behaviour also changed as the ramp angle increased. In the 5° and 10° files, the descent target was set back to zero:

```

case rampDescend:
    targetPosition = 0;
    integralCLAMPER = 19000;

    if (millis() - rampStartTime >= RAMP_WAIT_TIME) {
        rampState = rampComplete;
    }
    break;

```

In the 15° ramp file, the descent target was changed to 2000 counts first before setting it to 0:

```

case rampDescend:

```

```

        targetPosition = 2000;
        integralCLAMPER = 19000; //to slow the wind downhill

        if (millis() - rampStartTime >= RAMP_WAIT_TIME) {
            rampState = rampComplete;
        }
        Break;
case rampComplete:
    targetPosition = 0;
    integralCLAMPER = 6000; //help stop it
    rampState = rampIdle;
    break;

```

This made the downhill motion less aggressive. On a steeper ramp, immediately commanding the robot all the way back to zero could cause it to accelerate downhill too quickly. Using an intermediate target helped slow the descent and gave the controller more time to maintain balance.

Overall, the ramp development evolved from no ramp support in the midpoint code, to a general ramp state machine in the main final code, to separate ramp-tuned programs for 5°, 10°, and 15° testing. The main reason for using separate ramp files was time management. Since the ramp was developed late and by different coders, separating it from the main final code allowed the ramp logic to be tuned quickly without disrupting the rest of the final robot behaviour.

Several alternatives were considered. One option was to use the normal forward and backward commands to manually drive the robot up and down the ramp. This would have been simpler, but it would depend too much on operator timing and when we tried something similar, it did not work well and was highly inconsistent. The final ramp approach used state machines and ramp-specific parameters because the robot needed different behaviour for climbing, waiting, and descending.

The main lesson from ramp development was that ramp traversal is not just normal forward motion on an incline. The robot needs more lean authority while climbing, careful control while holding position, and reduced aggressiveness while descending. The separate 5°, 10°, and 15° ramp files show how the control parameters had to be adjusted as the incline increased.

Testing, Tuning, and Performance Analysis

Test 1: Motor Noise Test and Complementary Filter Tuning

One of the first tests performed during tuning was the motor noise test. The purpose of this test was to determine how the complementary filter constant k affected the filtered angle signal and whether running the motors introduced noticeable noise into the tilt-angle measurement. Since the balance controller depends directly on the filtered angle, it was important to choose a k value that provided a good balance between response speed and noise rejection.

The final code estimates tilt angle using both the accelerometer and gyroscope. The accelerometer angle is calculated from the measured y and z acceleration values, while the gyroscope value is corrected using the measured startup bias. The complementary filter then combines both values into one filtered angle:

```
accel_angle = atan2(y, z) * 180.0 / PI + angleOffset;
gyro_x -= gyroBiasX; // Remove bias
gyro_x = -gyro_x; //fix wrong direction of acceleration

unsigned long current_time = micros();
dt = (current_time - last_time) / 1000000.0;
last_time = current_time;

filtered_angle = k*(filtered_angle + gyro_x* dt) + (1-k)*accel_angle;
```

This implementation follows the complementary filter structure discussed in the lecture notes. In the final demo code, the filter constant was set to:

```
float k = 0.98;
```

This means that the filter places 98% weight on the gyroscope-based prediction and 2% weight on the accelerometer correction at each update. A lower k value gives more weight to the accelerometer, which allows the filter to correct quickly but makes the signal more sensitive to vibration and acceleration noise. A higher k value gives more weight to the gyroscope, which usually produces a smoother signal but increases the time required for the filter to correct gyro drift using the accelerometer.

Before comparing different k values experimentally, the time constant was calculated for several possible k values. The complementary filter time constant is:

$$\tau = \frac{k}{1-k} dt$$

where dt is the loop time. For the table below, $dt \approx 0.01s$ was used as an approximate loop time. Although the final code measures dt directly using micros(), using 0.01s gives a useful comparison between different k values.

Table 1: Meaning of complementary filter time constants.

Time (τ)	Meaning
1 time constant	About 63% corrected
3 time constants	About 95% corrected
5 time constants	Almost fully corrected

Table 2: Calculated complementary filter time constants for different k values

Filter constant, (k)	Accelerometer weight, (1-k)	Time constant, (τ)	Approx. (3τ) settling time	Approx. (5τ) settling time	Expected behaviour
0.50	0.50	0.010 s	0.030 s	0.050 s	Very fast response, but high accelerometer noise
0.70	0.30	0.023 s	0.070 s	0.117 s	Fast response, still sensitive to noise
0.90	0.10	0.090 s	0.270 s	0.450 s	Moderate response speed with some noise rejection
0.95	0.05	0.190 s	0.570 s	0.950 s	Smoother response, slower correction
0.98	0.02	0.490 s	1.470 s	2.450 s	Smooth signal with acceptable response time
0.99	0.01	0.990 s	2.970 s	4.950 s	Very smooth, but slow correction
0.999	0.001	9.990 s	29.970 s	49.950 s	Extremely slow correction, too much gyro dependence

From Table 2, lower k values such as 0.50 and 0.70 would respond very quickly, but they would rely heavily on the accelerometer. This is not ideal for a balancing robot because motor vibration and wheel motion can introduce noise into the accelerometer reading. On the other hand, very high values such as 0.99 or 0.999 produce a much smoother signal, but the filter becomes slow to correct gyro drift. Therefore, the most useful range for this robot was expected to be between approximately $k = 0.90$ and $k = 0.98$.

To experimentally compare the effect of motor noise, the filtered angle was recorded under four main conditions: $k=0.90$ with motors off, $k = 0.90$ with motors on, $k = 0.98$ with motors off, and $k = 0.98$ with motors on. These tests were designed to show both the effect of changing the complementary filter constant and the effect of motor vibration on the angle estimate.

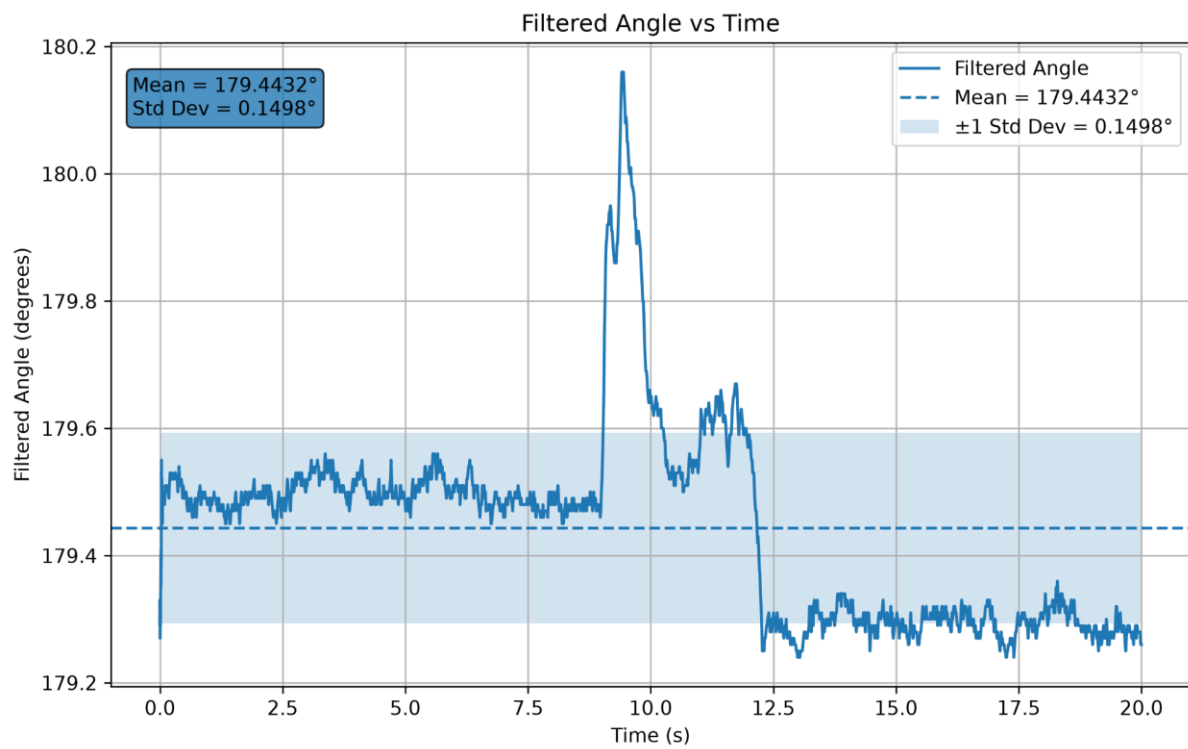


Figure 7: Filtered angle vs. time for $k = 0.90$ with motors off.

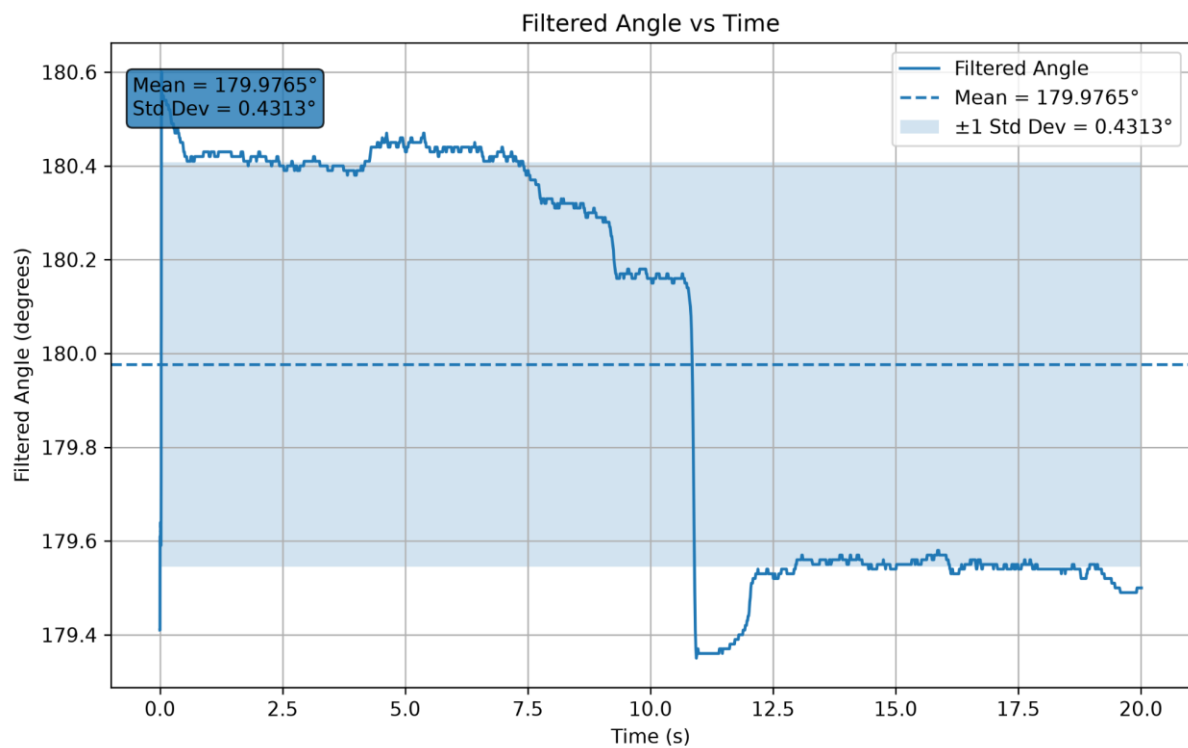


Figure 8: Filtered angle vs. time for $k = 0.90$ with motors on.

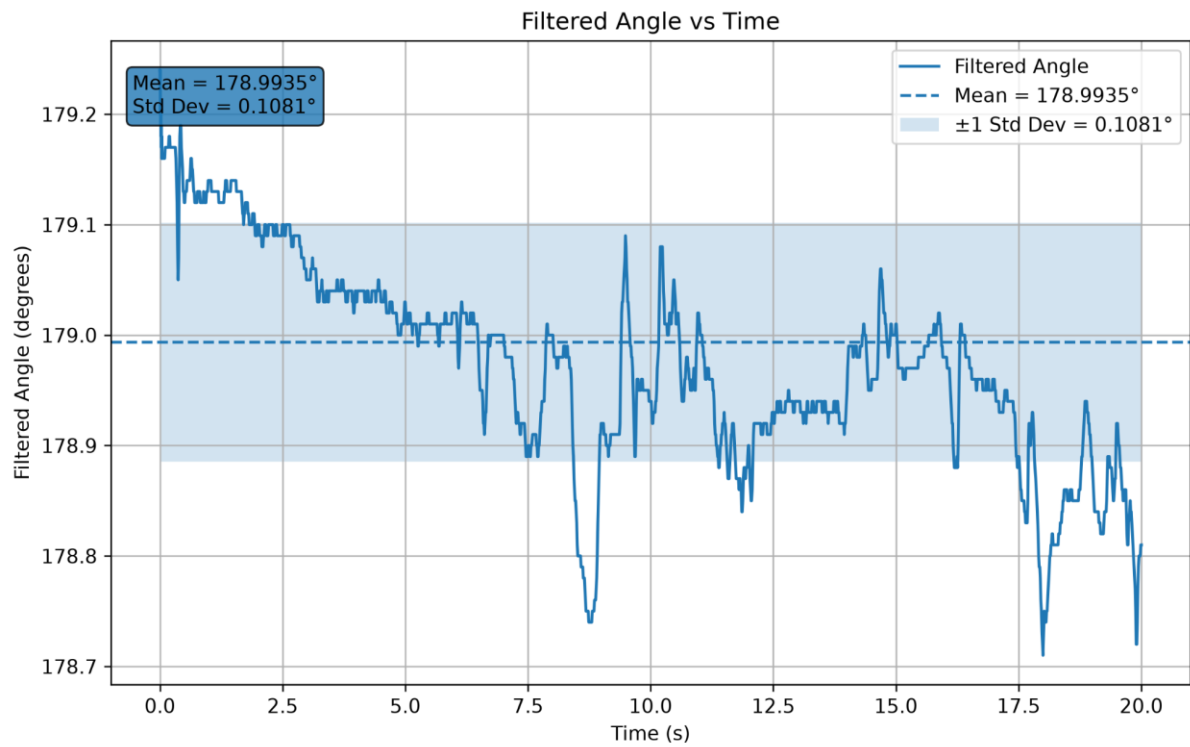


Figure 9: Filtered angle vs. time for $k = 0.98$ with motors off

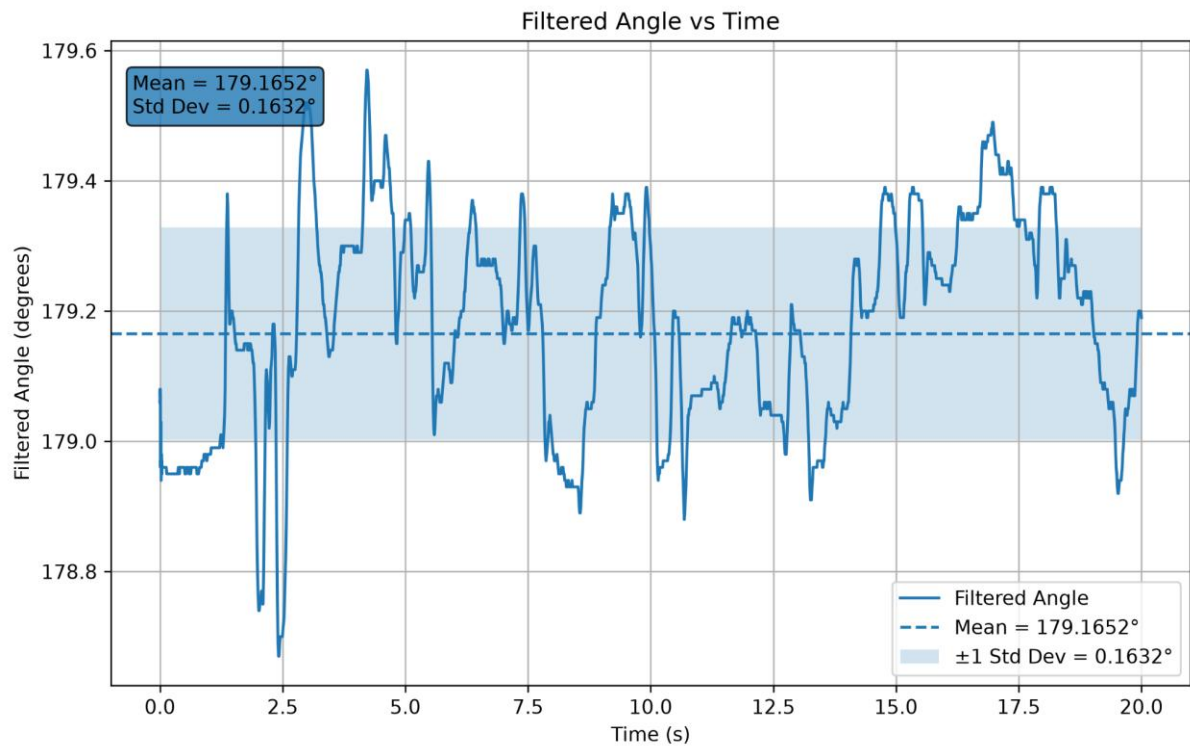


Figure 10: Filtered angle vs. time for $k = 0.98$ with motors on.

The measured noise values from the four plots were summarized using the standard deviation of the filtered angle. This provides a numerical way to compare the amount of variation in the angle signal for each test condition.

Table 3. Measured filtered-angle noise with motors off and motors on.

Filter constant, (k)	Motors off standard deviation	Motors on standard deviation	Change in noise	Observation
0.90	0.431°	0.148°	.283°	Faster response, but more visible angle variation
0.98	0.163°	0.108°	0.55°	Smoother filtered angle with better noise rejection

Comparing the plots and the standard deviation values shows the trade-off between response time and signal smoothness. The $k = 0.90$ case has a shorter time constant of approximately 0.09 s, so it can respond quickly to changes in angle. However, because it gives more weight to the accelerometer, it is also more affected by vibration and short-term noise. The $k = 0.98$ case has a longer time constant of approximately 0.49 s, meaning that it corrects more slowly, but it produces a smoother filtered angle.

Based on this comparison, $k = 0.98$ was selected for the final demo implementation. This value matched the final code and provided better noise rejection than $k = 0.90$, while still keeping the response time fast enough for the balancing controller.

Test 2: Single Loop PID Control Tuning

After completing the sensor filtering test and selecting a suitable complementary filter constant, the next stage of testing focused on tuning the single-loop balance PID controller. The goal of this test was to determine how the robot responded to changes in K_P , K_D , and K_I , and to find a range of values that produced stable upright behaviour. At this stage, the focus was not yet on accurate position control, turning, or ramp traversal. Instead, the purpose was to understand the basic balancing response of the robot when only the inner angle loop was being tuned.

The tuning method was mostly experimental. Rather than calculating exact gain values mathematically, each gain was tested by observing the robot's physical response. For each parameter, we first tested values that were clearly too low and values that were clearly too high. This helped identify the two extremes of the behaviour. After seeing these limits, the gains were adjusted gradually using trial and error until the robot showed a response that was fast enough to recover from tilt but not so aggressive that it oscillated or slammed into the ground.

The proportional gain, K_P , was tuned first because it had the largest effect on the robot's immediate response to tilt. When K_P was too low, the robot was not responsive enough. It could detect that it was falling, but the wheels did not accelerate quickly enough to move back under the center of mass. This caused the robot to slowly fall over instead of correcting itself. When K_P was too high, the opposite problem occurred. The motors reacted too aggressively to small angle errors, causing the robot to overshoot the upright position. In some tests, this overcorrection caused the robot to rapidly drive itself into the ground. Based on these observations, K_P was reduced until the robot still reacted strongly enough to recover from tilt, but without excessive oscillation.

After finding a usable proportional response, the derivative gain, KD, was tuned to add damping. The derivative term was important because it responded to how quickly the robot was rotating. When KD was too low, the controller had little damping, so the robot continued to overshoot after each correction. This made the robot look unstable even if the proportional gain was close to a usable value. When KD was too high, the robot became overly sensitive to fast changes and noise from the gyroscope. It could also damp the proportional response too much, making the robot feel less responsive even though KP was large enough. The final KD value was chosen as a compromise where the robot had enough damping to reduce overshoot, but not so much that it reacted harshly to noise or resisted the proportional correction. With just KP and KD, the robot was able to balance itself upright for a very brief duration before drifting to an irrecoverable angle.

The integral gain, KI, was tuned last because it accumulates error over time. This term was useful for correcting small long-term biases, such as a slight offset in the center of mass or a balance angle that was not perfectly vertical. When KI was too low, the robot could sometimes balance while leaning slightly forward or backward, which caused slow drift. When KI was too high, the accumulated error became too large and behaved like an increasing gain over time. This caused delayed overcorrection, where the robot might appear stable for a short time but then suddenly drive too strongly in one direction after the integral term built up. For this reason, the final integral gain was kept high enough to correct long-term bias, but low enough to avoid strong windup behaviour.

The main observations from the PID tuning process are summarized in Table 4.

Table 4: Observed effects during single-loop PID tuning.

Gain	Too low	Too high	Desired behaviour
KP	Robot was not responsive enough and fell before correcting	Oscillation, overcorrection, and aggressive motor response	Strong enough to recover from tilt without causing large overshoot
KD	Little damping, continued overshoot after correction	Overresponsive to noise and fast motion; could also make the robot feel overly damped	Enough damping to reduce oscillation without reacting too strongly to noise
KI	Robot could balance with a small lean or slow drift	Integral buildup caused delayed overcorrection and oscillation	Enough long-term correction to reduce bias without windup

The final single-loop PID gains used in the final demo code were:

```
const float KP = 8.25;    // 7.75
const float KD = 0.5;     // 0.5
const float KI = 45;     // 50
```

One important distinction from this test was that PID tuning alone did not completely solve hands-free balancing at first. The tuning process helped identify reasonable gain values and showed how each term affected the robot's behaviour, but the robot was still not able to reliably stand on its own until a later timing issue was fixed.

After the loop-frequency issue was corrected as described in the next section, the same general single-loop PID controller was able to balance much more reliably. At that point, the robot could remain upright using only the inner balance loop and stay close to the stationary balancing requirement. This showed that the PID gains were reasonable, but that controller timing was a necessary part of making the robot perform properly.

Overall, this test was useful because it established the relationship between the PID gains and the robot's physical behaviour. KP controlled the main corrective strength, KD provided damping, and KI corrected long-term lean or drift. The test also showed that tuning a self-balancing robot requires observing the real system, because motor response, chassis balance, sensor noise, and timing all affect whether the same gains will work in practice.

Test 3: Control Loop Frequency Test

After tuning the single-loop PID controller, the robot still did not balance reliably on its own. The PID tuning test showed that the gains were in a reasonable range, but the robot's response was still delayed and inconsistent. Based on feedback from the professor, the next test focused on the control-loop frequency. In class, it was recommended that the balancing loop should run at approximately 100–200 Hz. This was important because a self-balancing robot must correct its angle very quickly. If the loop runs too slowly, the robot receives sensor data, calculates the control output, and updates the motors too late. As a result, even well-tuned PID gains can behave poorly because the correction is always delayed.

To measure the loop frequency, a debug GPIO pin was toggled at the start and end of the main loop and measured using an oscilloscope. In the code, the debug pin was set high near the beginning of the loop and set low near the end:

```
const int DEBUG_GPIO = 4;
digitalWrite(DEBUG_GPIO, HIGH);
//loop code
digitalWrite(DEBUG_GPIO, LOW);
```

By measuring the period of this signal on the oscilloscope, the loop frequency could be calculated. This provided a direct measurement of how fast the robot was actually running its control loop, instead of only estimating the timing from the code.

The first oscilloscope measurement showed that the loop was running at approximately 50 Hz, which was much lower than the recommended 100–200 Hz range. This helped explain why the robot was difficult to tune. At 50 Hz, each loop took about 20 ms, meaning the robot could only update its sensor reading and motor command around 50 times per second. For a balancing robot, this delay is significant. The robot can continue falling noticeably during each loop interval, so the controller has less time to react.

The first suspected cause was the use of `Serial.print()` statements. Since serial communication can slow down an Arduino program, the debug print statements were removed or commented out during testing. However, the loop frequency did not improve significantly. This showed that serial printing was not the main bottleneck. After checking other parts of the code, the slowdown was found to be

caused mainly by repeated sensor and BLE checking inside the main loop. In particular, the code was checking IMU availability and BLE commands every cycle:

```
if (!IMU.accelerationAvailable())
if (!IMU.gyroscopeAvailable())
if (customCharacteristic.written())
```

These checks were useful, but performing them every loop added overhead and reduced the control-loop frequency. While the datasheet did not specifically make mention of the behaviour of the IMU and BLE functions, it is suspected that these functions are quite slow, and since the Arduino environment has no distinction of blocking vs nonblocking assignments, this means the entire loop is put on hold while the MCU waits for the response from the IMU and/or BLE. The BLE checking was especially unnecessary at the full balance-control rate because user commands do not need to be read as quickly as the angle controller runs. The angle loop needs to update very quickly, but BLE commands and some availability checks can be handled less frequently.

To improve the loop frequency, a loopsPassed counter was added so that slower checks were not performed on every loop. For example, BLE command checking and IMU availability checking were only performed on selected loop iterations:

```
if (central && (loopsPassed % 2 == 0)) {
    if (customCharacteristic.written()) {
    }
}

if (loopsPassed % 2 == 0) {
    if (!IMU.accelerationAvailable()) return;
    if (!IMU.gyroscopeAvailable()) return;
    loopsPassed = 0;
}

loopsPassed++;
```

This change reduced the amount of work done during every single control-loop cycle. Instead of allowing slower communication and sensor-checking tasks to limit the entire controller, the code separated the fast balancing calculations from slower background tasks. This allowed the main balancing loop to run more quickly and more consistently.

The oscilloscope was then used again to compare the loop frequency with and without the loopsPassed change. Although oscilloscope images from the earliest diagnostic test were not saved, oscilloscope readings from the final demo code were used to document the difference. The version without the loop-skipping approach produced a lower measured frequency, while the version with loopsPassed showed a faster loop rate closer to the recommended range.



Figure 11: Oscilloscope measurement of before the loopsPassed optimization.

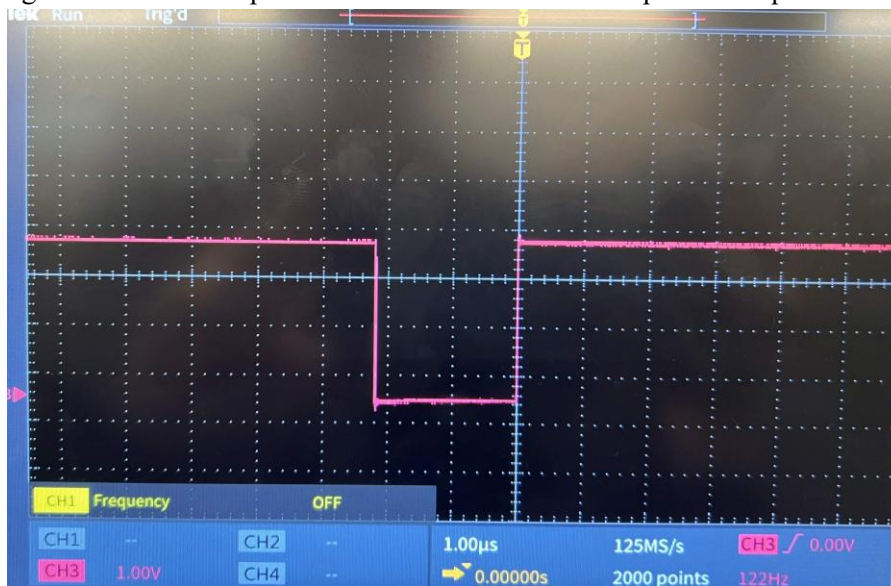


Figure 12: Oscilloscope measurement of before the loopsPassed optimization.

Our observations from the loopsPassed optimization are shown in Table 5.

Table 5: Control-loop frequency measurements.

Test condition	Measured loop frequency	Observation
Initial loop-frequency test	Approximately 50 Hz	Much lower than the recommended 100–200 Hz range
Serial print statements removed	Approximately 54 Hz	Little improvement, so serial printing was not the main cause
Final demo code without loopsPassed	Approximately 54 Hz	Loop still ran slower because BLE and sensor checks occurred too often
Final demo code with loopsPassed	Approximately 122 Hz	Loop frequency improved and balancing became more reliable

This test was important because it showed that the remaining balancing issue was not only caused by PID gain values. The previous PID tuning test had identified reasonable values for K_P , K_D , and K_I , but the controller could not perform properly when it was running too slowly. After the loop-frequency issue was improved, the robot was able to self-balance much more reliably using the same general single-loop PID controller. This was the point where the robot was able to remain upright on its own and stay close to the stationary balancing requirement.

The main design decision from this test was to avoid running slower tasks at the same rate as the balance controller. BLE communication, debug printing, and repeated availability checks do not need to occur as quickly as the angle correction loop. By reducing how often those checks were performed, the robot could update its balance control more frequently. This improved the response time of the controller and made the PID gains behave more consistently.

Altogether, the frequency test showed that real-time timing is just as important as controller tuning for a self-balancing robot. A set of PID gains that appears unstable at a low loop frequency may work much better once the loop runs fast enough. The oscilloscope measurement was useful because it provided direct evidence of the actual control-loop rate and helped identify why the robot was not balancing reliably even after PID tuning.

Challenges and troubleshooting

Challenge 1: Hardware Damage

One of the most significant challenges encountered during the project was accidental hardware damage caused by an unsafe startup condition. During one test, a newly uploaded program caused the motors to drive at maximum PWM immediately on startup. Because the robot was powered and positioned on the table at the time, it suddenly launched itself off the edge and fell to the floor. The impact damaged both a wheel assembly and the Arduino board. One of the wheels had its spokes damaged, while the Arduino had its micro-USB port physically torn off.



Figure 13: Damaged Wheel after Startup Crash

The observed problem was therefore not only a software issue, but also a mechanical and electrical failure that directly affected the rest of the project. After the crash, the robot could no longer be used reliably in its existing condition. The damaged wheel introduced wobble and mechanical

inconsistency, which affected the smoothness of motion and balancing performance. At the same time, the damaged Arduino could no longer be programmed or powered through its USB connection, making it unusable for further development.

The suspected cause of the problem was that the code allowed the motors to receive a very large command immediately at startup. In other words, the control output was not safely initialized before the robot began executing the balancing code. Since the battery pack was still connected while the code was being uploaded and tested, the robot was able to apply motor power as soon as the program started. This created a dangerous condition where any incorrect initialization, unstable starting angle, or unexpected PWM output could cause the robot to move suddenly. In this case, the startup PWM appears to have saturated to a very large value, causing the robot to accelerate violently before it could be stabilized.

The troubleshooting process first involved identifying which components had been damaged and determining whether they could still be used. Inspection showed that the wheel was mechanically compromised and no longer rotated properly, while the Arduino had suffered permanent damage to the USB connector. Since both of these components were critical to the robot, the immediate solution was therefore to replace the damaged hardware as quickly as possible. A new Arduino was purchased, and an email was sent to request a replacement wheel. Fortunately, both replacement parts were obtained in time for the remainder of the project.

Although replacing the parts restored the robot's functionality, the new hardware did not behave exactly the same as the old hardware. After installing the replacement Arduino and wheel, the robot no longer responded identically to the previously tuned controller. As a result, some slight retuning was required. In particular, the PID gains and the motor deadband values had to be adjusted again so that the robot could balance and move properly with the replacement components. This showed that even when the design is nominally the same, small changes in hardware can affect controller performance enough to require additional tuning.

The final solution to this challenge had two parts. First, the damaged hardware was replaced so that development could continue. Second, a safer testing procedure was adopted to prevent the same failure from happening again. After this incident, the battery pack was always turned off before uploading new code or making major software changes. This reduced the chance of the motors unexpectedly activating as soon as the robot restarted. More generally, the team became much more careful about startup conditions and the possibility of unsafe motor outputs during initialization.

This challenge had an important influence on the final design process because it showed that reliable robot development depends not only on good control code, but also on safe testing procedures. The main lesson learned was that a self-balancing robot should always be treated as an active electromechanical system that can move suddenly if the software behaves unexpectedly. In future work, this risk could be reduced further by including a software arming condition, startup safety lock, or requirement that the motors remain disabled until a valid operating condition is confirmed. Even though the hardware was replaced successfully, this incident emphasized the importance of both physical caution and defensive software design during robot development.

Challenge 2: Unequal Wheel Speed

Another significant technical challenge was that the two wheels did not move at exactly the same speed, even when they were given the same PWM command. This caused the robot to drift during straight-line movement and made turning less consistent. Ideally, equal motor commands should produce similar wheel speeds, but in practice the left and right motors had different friction, deadband, and response characteristics. This became especially noticeable when the robot was trying to move straight or perform controlled turns, because small differences between the two wheels could gradually rotate the robot away from its intended direction.

The suspected causes were mechanical and electrical differences between the two motor assemblies. Even though the two motors were the same model, their actual behaviour was not identical. The gearboxes, wheel friction, motor drivers, wiring, and physical load on each wheel could all cause slightly different responses. The issue was also made more complicated by the earlier hardware damage and replacement parts, since the robot's wheel behaviour changed after the damaged components were replaced. As a result, the previously tuned motor values did not perfectly match the new hardware.

At first, the planned solution was to automatically tune the motor deadband during startup. The idea was that the robot could slowly increase the PWM command to each motor and use the encoders to detect when the wheel first began moving. This value would then be stored as the deadband for that motor. The final code still contains this `autoDeadband()` function, but it was commented out and not used in the final demo.

The original purpose of this function was to avoid relying on fixed deadband values, since the motor deadband could change slightly depending on battery level, surface friction, or hardware condition. In principle, this would make the robot more adaptive. However, during testing, the robot did not behave as well when this automatic deadband method was used. The startup calibration did not consistently produce values that improved balancing or movement, so it was removed from the active final code. One reason the automatic deadband approach was not reliable was that the deadband measured at startup did not fully represent the robot's behaviour during balancing. The test only detected the PWM value where each wheel first began to move, but the robot's actual operating conditions were more dynamic. During balancing, the motors constantly changed direction and speed, and the wheels experienced changing loads as the robot tilted forward and backward. A startup deadband value based only on initial encoder movement was therefore not enough to describe the full motor response. In addition, running motor tests during startup made the robot's initialization more complicated and introduced another possible source of inconsistent behaviour.

To better understand the wheel mismatch, a separate encoder-count test was added in the setup code for debugging. Both motors were commanded to run at the same PWM value for a fixed time window, and the encoder counts from each wheel were compared:

```
setLeftMotor(250);  
setRightMotor(250);  
delay(2000); // let them stabilize  
// Read encoder counts over a fixed time window  
noInterrupts();
```

```

    long L0 = LEncCount;
    long R0 = REncCount;
    interrupts();
    delay(2000);
    noInterrupts();
    long L_count = LEncCount - L0;
    long R_count = REncCount - R0;
    interrupts();
    setLeftMotor(0);
    setRightMotor(0);
    computeEncoderValues();
    Serial.print("L counts: "); Serial.print(L_count);
    Serial.print("  R counts: "); Serial.println(R_count);
    Serial.print("Ratio (L/R): "); Serial.println((float)L_count /
R_count);

```

This test showed that the mismatch between the left and right motors was not constant across all PWM values. The measured left-to-right encoder-count ratio changed depending on the applied PWM.

Table 6. Motor imbalance observations from encoder-count testing.

PWM command	Measured L/R encoder-count ratio	Observation
60	1.02–1.08	Left wheel was slightly faster
80	1.07–1.09	Left wheel was consistently faster
150	0.96	Wheels were closer, with right slightly faster
250	0.77–0.83	Right wheel became much faster at high PWM

These results showed that the motor imbalance was nonlinear. A single correction factor could not perfectly fix the mismatch at every speed, because the wheel-speed ratio changed depending on the PWM range. This also helped explain why the automatic deadband method did not solve the problem. The issue was not only the starting PWM threshold; the two motors also responded differently once they were already moving.

The final solution was to keep manually tuned deadband values and add a simpler empirical imbalance correction. The final code used fixed deadband compensation values:

```

int L_DEADBAND = 42;
int R_DEADBAND = 42;

```

To help compensate for remaining left-right mismatch, the final code also used an `imbalanceMultiplier` on the right motor command:

```
int leftPWM = applyDeadband(basePWM + headingCorrection, L_DEADBAND);
int rightPWM = applyDeadband((basePWM -
headingCorrection)*imbalanceMultiplier, R_DEADBAND);
```

The multiplier was not intended to perfectly model the motor behaviour at every PWM value. Instead, it was a practical tuning adjustment that improved the robot's real movement and turning behaviour. During normal operation, the multiplier could remain at 1.00, while during turning it was adjusted to 1.07.

This value was chosen experimentally because it improved the turning response without overcorrecting too much. Since the encoder-count test showed that the motor ratio changed significantly with PWM, using a very large multiplier based on high-PWM behaviour would not have worked well at lower PWM values. A smaller empirical correction was therefore more useful for the actual balancing and turning conditions.

The main lesson from this challenge was that motor mismatch cannot always be fixed by a simple startup calibration. The deadband value only describes the PWM threshold needed to start motion, but it does not fully describe how the motors behave once they are moving under load. The encoder measurements showed that the two wheels had different responses at different PWM levels, so the final solution had to be based on practical testing rather than a single automatic calculation.

This challenge influenced the final design by keeping the motor compensation simple and robust. Instead of relying on `autoDeadband()`, the final robot used fixed deadband values, heading correction, and a small imbalance multiplier.

Challenge 3: Z-Axis Gyroscope Bias

The third significant challenge was caused by drift in the z-axis gyroscope measurement. The z-axis gyroscope was used to estimate the robot's heading during turning by integrating angular velocity over time. Ideally, when the robot was stationary, the z-axis gyroscope reading should have been close to zero. However, in practice, the raw `gyro_z` value had a small bias. Even though this bias was small, it became important because the heading estimate was calculated by integration. A small constant error in angular velocity could accumulate over time and make the robot think it had turned even when it had not.

The observed problem was that the robot's stability and self-balancing behaviour became worse when the startup z-axis gyro calibration value was used. In the code, the function `calibrateGyroX()` also included a calibration calculation for the z-axis gyroscope. During startup, the robot collected multiple gyroscope samples and averaged the z-axis values:

```
sumZ += gz;
gyroBiasZ = sumZ / samples;
```

The original intention was to subtract this startup bias from the z-axis gyroscope reading during operation. In theory, this should have reduced heading drift. However, during testing, the startup-calibrated value was not consistent enough, and using it made the robot behave worse. The robot would sometimes apply unwanted heading correction even when it was not supposed to be turning. This interfered with the balance controller because the motor outputs included both the main balance PWM and the heading correction term.

The suspected cause was that the startup calibration was sensitive to small variations in the initial condition. Even if the robot appeared stationary, it was difficult to guarantee that it was perfectly still every time the code started. Small vibrations, slight hand movement, surface tilt, sensor noise, or timing differences during startup could change the averaged gyroBiasZ value. Since the heading estimate depends on integrating gyro_z, even a small error in the bias value could cause noticeable heading drift over time.

The troubleshooting process involved comparing the startup-calibrated bias with a more controlled stationary measurement. To do this, the robot was left stationary and approximately 500 raw z-axis gyroscope readings were recorded. These values were then transferred into an Excel spreadsheet, where the average raw gyro_z value was calculated. This average represented the measured stationary bias of the z-axis gyroscope. The value found from this test was 0.149990639 and was then used as a fixed offset in the code:

```
gyro_z = gyro_z-0.149990639;
```

After subtracting this experimentally measured offset, the z-axis heading drift was reduced. This helped prevent the heading correction system from activating unnecessarily while the robot was trying to balance or move straight. The corrected gyro_z value was then integrated to update the heading estimate:

```
zAngle += gyro_z*dt;
```

Although this fixed-offset method was very similar in concept to the startup calibration method, it produced more consistent results during testing. The main difference was that the Excel-based value was measured during a controlled stationary test and did not change every time the robot started. In contrast, the startup calibration could produce slightly different values depending on the exact startup condition. Because the fixed value gave better real-world performance, it was kept in the final code instead of using the automatically calculated gyroBiasZ.

The final solution was therefore to use the experimentally measured z-axis gyroscope offset as a constant correction value. This was not the most general solution, but it was more reliable for our specific robot during the final testing period. It reduced heading drift and made the turning and straight-line heading correction more stable. Since the robot's heading correction affected the left and right motor commands, improving the gyro_z bias correction also helped prevent unnecessary differential motor output during balancing.

This challenge showed that sensor calibration methods must be evaluated based on actual system performance, not only theory. A startup average should have worked in principle, but in practice it was too sensitive to small disturbances and produced less consistent behaviour. The main lesson was

that gyroscope bias is especially important when the signal is integrated, because small errors can accumulate into large heading errors.

Conclusions

The final self-balancing robot was able to demonstrate the main functions required for the project, including upright balancing, forward and backward motion, controlled turning, BLE-based remote commands, and ramp traversal testing. The robot used the Arduino Nano 33 BLE Sense Rev2 as the central controller, with the onboard IMU providing tilt feedback and the wheel encoders providing motion feedback. Through repeated testing and tuning, the robot was able to maintain balance, respond to movement commands, and use feedback from both the IMU and encoders to improve its stability and motion control.

A major technical achievement of the project was the development of a working cascade control system. The inner loop used the filtered IMU angle to stabilize the robot, while the outer loop used encoder position feedback to command a target lean angle. This allowed the robot to move toward a target position while still maintaining fast balance correction. Additional software features, such as BLE command handling, heading correction using the z-axis gyroscope, target smoothing, deadband compensation, and ramp-specific state machines, were added to improve the robot's performance for the final demo tasks.

The project also showed the importance of experimental testing in robotics. Several design decisions were guided by real measurements and observations rather than theory alone. Complementary filter tuning improved the angle estimate, PID tuning helped identify stable gain ranges, and oscilloscope testing showed that the control-loop frequency was too low before optimization. Troubleshooting also revealed practical hardware and software challenges, including motor mismatch, wheel deadband, gyroscope bias, startup safety, and the effect of damaged or replaced hardware on controller tuning.

One of the most important lessons learned was that a self-balancing robot depends on the entire system working together. Stable performance was not achieved by changing only one gain or one component. Instead, the final result required reliable sensor filtering, consistent loop timing, safe motor control, mechanical alignment, encoder feedback, and careful tuning. The project also emphasized the need for safe testing procedures and safety practices. This project ultimately provided a strong practical understanding of feedback control, embedded programming, sensor fusion, and real-world troubleshooting in an unstable robotic system.

Future Improvements

Although the final robot was able to complete the main demo objectives, there are several realistic improvements that could further improve its performance, consistency, and reliability.

One important future improvement would be to add a closed-loop wheel-speed balancing controller between the left and right motors. In the final design, wheel mismatch was handled using fixed deadband values and an experimentally tuned imbalance multiplier. This was effective enough to reduce visible drift and improve turning consistency, but it did not fully correct the fact that the two wheels responded differently at different PWM values. A better solution would be to continuously

compare the left and right encoder speeds while the robot is moving and automatically adjust the motor commands to keep both wheels synchronized. This would allow the robot to drive straighter and turn more consistently, especially as the battery voltage, floor friction, or motor loading changes.

Another improvement would be to make the z-axis gyroscope calibration more robust. In the final code, a fixed experimentally measured offset was subtracted from `gyro_z` because the startup calibration produced inconsistent results. This worked better for the final demo, but it was still specific to the robot and testing conditions at that time. A more reliable approach would be to perform z-axis gyro calibration only after confirming that the robot is completely stationary. For example, the code could check that the accelerometer and gyroscope readings remain within a small threshold for a short period before accepting the calibration value. This would reduce the chance of using a biased startup measurement caused by small vibrations, hand movement, or table motion. In future work, the z-axis gyroscope could also be combined with another heading reference to reduce long-term drift during turning.

A third improvement would be to better integrate the ramp-control code into the main final program. Because ramp development began late, the ramp behaviour was tested in separate 5°, 10°, and 15° ramp files. This allowed the team to tune the ramp quickly, but it also meant that the ramp code was not as cleanly integrated with the main balancing, position, and turning code. In future work, the ramp logic could be combined into one unified state machine with selectable ramp modes or adaptive parameters. This would make the software easier to maintain and would allow the same program to handle flat-ground motion, turning, and ramp traversal without switching between separate code files.

Ultimately, the most useful improvements would focus on making the robot more adaptive. The final design relied on several experimentally tuned constants, such as motor deadband, imbalance multiplier, gyro offset, and ramp-specific parameters. These values worked for the final hardware and testing conditions, but a more advanced version of the robot could measure and adjust some of these values automatically. This would make the robot more reliable across different surfaces, battery levels, ramp angles, and hardware variations.

References

- [1] Arduino, “Arduino Nano 33 BLE Sense Rev2 Datasheet,” Arduino Documentation, 2024. [Online]. Available: <https://docs.arduino.cc/resources/datasheets/ABX00069-datasheet.pdf>. [Accessed: Jun. 17, 2026].
- [2] Texas Instruments, “DRV8871 3.6-A Brushed DC Motor Driver With Internal Current Sense (PWM Control),” DRV8871 Datasheet, Rev. B, Mar. 2019. [Online]. Available: <https://www.ti.com/lit/gpn/DRV8871>. [Accessed: Jun. 17, 2026].
- [3] Pololu Corporation, “30:1 Metal Gearmotor 37Dx68L mm 12V with 64 CPR Encoder (Helical Pinion),” Pololu Product #4752. [Online]. Available: <https://www.pololu.com/product/4752>. [Accessed: Jun. 17, 2026].
- [4] Arduino, “ArduinoBLE Library,” Arduino Documentation. [Online]. Available: ArduinoBLE official documentation. [Accessed: Jun. 17, 2026].

[5] ELEC 391 Course Staff, “Lecture 1: Sensors, IMU, and Complementary Filtering,” ELEC 391 Course Notes, University of British Columbia, 2026.

[6] ELEC 391 Course Staff, “Lecture 2: Motors, Motor Drivers, PWM, and Encoders,” ELEC 391 Course Notes, University of British Columbia, 2026.

[7] ELEC 391 Course Staff, “Lecture 3: Control, Inverted Pendulum Dynamics, and PID Control,” ELEC 391 Course Notes, University of British Columbia, 2026.

[8] ELEC 391 Course Staff, “Lecture 4: Tips for Robot Integration, Filtering, Timing, Deadband, and PID Tuning,” ELEC 391 Course Notes, University of British Columbia, 2026.

Appendices

Appendix A: Encoder Voltage Divider Calculations

The voltage divider values were chosen using the equation : $V_{out} = V_{in} \frac{R_2}{R_1 + R_2}$, where V_{out} was the reduced voltage sent to the arduino logic pins, and V_{in} was the 12V input from the battery pack.. Since the Arduino uses 3.3 V logic, the target output voltage was approximately 3.3 V. However, we intentionally chose resistor values that would slightly undershoot 3.3 V for safety reasons. This was done to account for resistor tolerance and to reduce the risk of accidentally applying a voltage above the safe logic level of the Arduino.

Instead of using exact resistor values, we derived suitable values based on the most commonly available resistors in the ELEC second-year kit. Using $R_1 = 10\text{ k}\Omega$ and $R_2 = 3\text{ k}\Omega$, the expected output voltage calculated on output voltage of approximately $V_{out} = 12 * \frac{3}{10+3} \approx 2.78\text{ V}$

After building the circuit, we measured V_{out} and found that it was not exactly 3.3 V. Instead, the measured output voltage was approximately 2.3 V. Although this was lower than the target value, it was still sufficient for the Arduino to read as a valid logic signal in our setup.

Appendix B: Final Pin Connections

The final pin connections used in the main demo code are shown in Table B.1.

Table B.1. Arduino pin connections used in the final robot.

Component	Signal	Arduino pin	Purpose
Debug output	DEBUG_GPIO	D4	Used to measure loop frequency with an oscilloscope
Left motor driver	L_IN1	D5	Left motor direction/PWM control

Left motor driver	L_IN2	D6	Left motor direction/PWM control
Right motor driver	R_IN1	D9	Right motor direction/PWM control
Right motor driver	R_IN2	D10	Right motor direction/PWM control
Left encoder	L_ENC_A	D7	Left encoder channel A
Left encoder	L_ENC_B	D8	Left encoder channel B
Right encoder	R_ENC_A	D11	Right encoder channel A
Right encoder	R_ENC_B	D12	Right encoder channel B

Both motor drivers were powered from the robot battery pack, while the Arduino provided the low-power logic signals used to control the DRV8871 inputs. The encoder signals were connected to the Arduino through voltage dividers so that the signal levels remained within the safe input range of the Arduino Nano 33 BLE Sense Rev2. All electrical subsystems shared a common ground so that the Arduino, motor drivers, encoders, and battery pack had the same voltage reference.

Appendix C: Final Control Gains and Parameters

The final demo code used experimentally tuned control gains and constants. These values were adjusted through testing so that the robot could balance, move forward and backward, turn, and respond to BLE commands. The main final control parameters are summarized in Table C.1.

Table C.1. Main final control gains and parameters.

Parameter	Final value	Purpose
PWM_MAX	255	Maximum motor PWM command
KP	8.25	Angle proportional gain
KD	0.5	Angle derivative gain
KI	45	Angle integral gain
PC_KP	0.0015	Position proportional gain
PC_KD	0.00075	Position derivative gain initialized in the main code
PC_KI	0.0003	Position integral gain
k	0.98	Complementary filter constant
targetAngle	-0.7°	Default balance target angle

MAX_ANGLE	3.0°	Maximum target lean angle during normal flat-ground motion
PC_LOOP_DELAY	40 ms	Position loop update period
TARGET_SLEW_RATE	900 counts/s	Limits how quickly the target position changes
L_DEADBAND	42	Left motor deadband compensation
R_DEADBAND	42	Right motor deadband compensation
CONTROL_SIGN	-1	Corrects motor direction relative to measured tilt
imbalanceMultiplier	1.00 normally, 1.07 during turning	Empirical correction for unequal wheel response
gyro_z offset	0.149990639	Experimentally measured z-axis gyroscope bias correction
Heading tolerance	±2°	Allowed heading error before correction is applied

The position derivative gain was adjusted during testing, and the final code also restores some position-control values during the ramp idle state. For this reason, the values in this table should be interpreted as the main final-demo tuning values used by the robot rather than theoretical design constants. The gains were selected experimentally based on balancing stability, movement smoothness, turning response, and ramp performance.

For ramp testing, separate code files were used for 5°, 10°, and 15° ramp trials. These files used different ramp-specific values for MAX_ANGLE, TARGET_SLEW_RATE, target position, and integral limiting. The ramp files are included with the submitted code.